LEAL

Para la línea de computadoras CID se creó un lenguaje orientado al cálculo científico denominado LEAL (LEnguaje ALgorítmico), logrando resultados positivos en su utilización. Fue un auto-código de diseño propio cubano, en notación polaca y simbólica, que seguía algunos de los conceptos del Autocode de la Elliot 803B Mark III. Creado para la CID 201-A en 1970-71 por los especialistas de programación del CID, principalmente el especialista principal Víctor Llopis Zayas-Bazán, el LEAL permitió un mayor y mejor uso de la computadora y muchos de los que aprendieron computación en los años 70´ del pasado siglo utilizaron este sencillo lenguaje.

Con el avance de la línea de máquinas CID, el lenguaje LEAL fue evolucionando paralelamente. Se desarrollaron versiones específicas para cada modelo de la serie CID: el LEAL-A para la CID 201-A, el LEAL-B para la CID 201-B y el LEAL-F para la CID 300/10 (aunque también se empleó en la 201-B).

La versión LEAL-A se destaca por su especialización en cálculos científicos. El LEAL-B, más evolucionado, ofreció un rango más amplio de aplicaciones dentro de este campo, mejoró las subrutinas, la gestión de nuevos periféricos y sistemas de entrada/salida, así como introdujo la posibilidad de incluir comentarios, lo que mejoraba significativamente la legibilidad y mantenibilidad del código.

Por su parte, la versión LEAL-F combina capacidades avanzadas para cálculos científicos con funcionalidades adicionales para la gestión y cálculo económico, incorporando el tratamiento de caracteres tanto a través de programación personalizada como mediante subrutinas estándar.

El LEAL-F se utilizó un corto tiempo con la CID-300 al inicio de su explotación, para lo que hubo de desarrollarse ejecutores, así como traductores de código, programas y datos desde la CID 201-B, pero fue pronto abandonado en favor de lenguajes más avanzados, estandarizados y que explotaban mejor el nuevo equipamiento y los sistemas operativos. Un estudio del LEAL-F determinó que la mayoría de los programas de cómputo científico-técnico empleados en Cuba contaban de entre 30 y 120 líneas de código, aunque llegando algunos a 500 líneas.

El lenguaje LEAL encontró una amplia aplicación entre los usuarios de la familia de máquinas CID, tanto en Cuba como en otros países donde se utilizaron estos equipos. Aunque su diseño está orientado principalmente al cálculo científico, también se empleó en proyectos relacionados con la gestión económica y combinaciones de ambos ámbitos. Ejemplos notables incluyeron sistemas de control de producción en industrias textiles, bases de datos de reparaciones, sistemas de gestión de fuerza laboral en instituciones agropecuarias, trabajos de planificación en empresas del sector de transporte y sistemas PERT desarrollados localmente, entre otros.

Elementos del Lenguaje LEAL

Los elementos que componen el lenguaje LEAL son seleccionados por el programador a partir de conjuntos fijos y deben ser adheridos a reglas estrictas, lo que impide su modificación, adición o reordenamiento. Estos elementos incluyen instrucciones redactadas en código simbólico, símbolos de puntuación, operadores aritméticos y condiciones, así como letras del alfabeto que identifican variables.

Los programas fuente en LEAL se elaboraban generalmente en hojas especiales que facilitan tanto al programador como al operador la visualización y organización del código. Dado que LEAL utilizaba en principio cinta de papel perforado, cada instrucción se registraba en una línea separada, delimitada por el símbolo de retorno de carro. Estas hojas incluían una columna destinada a los números de referencia (etiquetas de salto e instrucciones) y un espacio para que el programador agregue notas explicativas. Aunque estas anotaciones del programa no podían ser perforadas en cinta, eran y son cruciales para la revisión y optimización del mismo.

Los caracteres válidos son: Letras de la A a la Z, dígitos del 0 al 9, caracteres especiales: + — : / = espacio ( ) . . ’ ? $.

Un programa en LEAL se estructura en dos partes principales:

Encabezamiento del Programa: En esta sección se incluye el título del programa, la declaración de variables utilizadas — especificando cuáles son enteras y cuáles flotantes (números con punto decimal)— y se hace constar el número de referencia (etiqueta de instrucción) más alto utilizado en el programa.

Cuerpo del Programa: El cuerpo contiene las instrucciones del programa organizadas en el orden de ejecución. Estas instrucciones se agrupan en varias categorías: asignación y aritméticas, entrada/salida, especificación, control y subrutinas.

Instrucciones|

Instrucciones de asignación y aritméticas: Estas instrucciones permiten realizar operaciones aritméticas derivadas del análisis matemático. Sin embargo, son más limitadas en comparación con las capacidades de FORTRAN, ya que LEAL no permite el uso de paréntesis en las operaciones aritméticas, operando siempre de izquierda a derecha hasta llegar al símbolo de asignación. Como resultado, una instrucción compleja en FORTRAN puede requerir varias líneas en LEAL.

Funciones: El LEAL ofrece aproximadamente doce funciones aritméticas, mientras que FORTRAN cuenta con una gama más amplia. No obstante, las funciones disponibles en LEAL constituyen un conjunto robusto y efectivo para aplicaciones de cálculo científico.

Instrucciones de entrada/salida

El lenguaje LEAL, diseñado para facilitar operaciones de entrada/salida, exige la identificación previa de los dispositivos que se emplearán, así como de las variables asociadas a la lectura y escritura de datos, y el formato de estos (ya sea entero, punto flotante o en notación editada).

Los dispositivos de entrada/salida que se incorporaron a las computadoras o se desarrollaron a su par requerían una coordinación entre las acciones del operador y las instrucciones correspondientes en el código, como era el caso de las grabadoras en cinta TEAC y Sanyo empleadas en las CID 201/201-A. Esto se fue resolviendo al incorporar mejores periféricos, totalmente controlados por programa.

Las variables dentro del programa están vinculadas a un número fijo de localizaciones (máquina a palabra), y su edición admite tres formatos distintos: entero, con punto decimal y exponencial, todos dentro de la precisión que ofrece el lenguaje.

En términos generales, se pueden distinguir dos categorías de instrucciones: aquellas dedicadas a la lectura y escritura, y las que se encargan del formato de salida. El tratamiento de caracteres en LEAL, especialmente en lo que respecta al manejo de archivos, se mantuvo en constante evolución durante el tiempo en que el lenguaje estuvo en uso. Por ejemplo, se le incluyó una instrucción para transferir cadenas de caracteres alfanuméricos entre periféricos y la memoria, sin manipularlos.

Una vez que el programa LEAL ha sido perforado en cinta, el cuerpo del programa establece el orden secuencial de ejecución de las instrucciones, comenzando por la primera y avanzando linealmente, salvo que una proposición de control modifique este flujo.

Las instrucciones que impactan el control del programa son los s**altos incondicionales**, que permiten cambiar el flujo sin condiciones previas; los s**altos condicionados**, basados en el cálculo del valor de una expresión aritmética; los c**iclos, q**ue ejecutan un bucle mientras se controlan las variables de conteo, incremento y control; y las instrucciones de alto y finalización del programa.

Instrucciones de subrutina

La ejecución de un conjunto de instrucciones de manera repetitiva, constituyendo una operación autónoma dentro del programa principal, se logra en LEAL mediante las instrucciones de llamada y retorno de subrutinas. Este lenguaje presenta dos enfoques clásicos para la implementación de subrutinas: el primero implica la transferencia de valores desde un conjunto variable del programa principal hacia las variables de la subrutina, con un proceso inverso al finalizar; el segundo enfoque permite que la subrutina opere directamente sobre los mismos valores utilizados en el programa principal, sin necesidad de realizar ninguna transferencia.

Para representar variables se puede emplear cualquier letra del alfabeto, excepto la ñ, en mayúscula. Las constantes aparecen en forma decimal, y se tratan como enteros o flotantes según la variable a al que se asocie en una operación.

Para simplificar el texto siguiente se tomará esta convención:

REPRESENTACION EN EL TEXTO SIGUIENTE	
A, B, C, D, …	representan variables flotantes
I, J, K, L, …	representan variables enteras
l, m, n, …	repres. constantes enteras positivas
x, y, z, …	repres. const. ent. o flot. Positivo

Los sufijos permiten designar con una misma letra, a un grupo de variables que tienen cierta similitud, dígase, por ejemplo, un arreglo de valores. Las letras o variables con sufijos son tratadas por el compilador como variables diferentes. Cualquier número entero positivo o cero, o una variable entera positiva puede usarse como sufijo de cualquier tipo de variable. Los sufijos se encierran entre paréntesis. Ejemplos de sufijos: A(0), A(n), A(I), A(I(n)).

Los paréntesis sólo son permitidos para encerrar sufijos y números de referencia, para observaciones y chequeos. No se permite su uso para señalar prioridad en las operaciones.

Los números de referencia sirven como etiquetas identificativas de una línea de código. Los números de referencia aparecen al inicio de la línea, a la izquierda y entre paréntesis. Deben ser siempre constantes enteras y positivas y no se permite utilizar el mismo número de referencia más de una vez. Ejemplo de número de referencia: (1) A + B = C.

Para las variables se pueden usar letras mayúsculas del alfabeto (excepto la ñ) como variables. El programador define si son enteras o flotantes, y esa condición no cambia durante el programa. Cualquier intento posterior de modificar el tipo será error de compilación. Se permiten subíndices.

Las constantes se escriben en forma decimal y se interpretan según el tipo de la variable asociada. Si una constante entera se usa con una variable flotante al inicio de línea, debe escribirse con punto y un cero.

Ejemplos	
A+4=B	si A y B son flotantes, el 4 se trata como flotante
I+4=J	si I y J son enteros, 4 se trata como entero
4.0+A=B	constante flotante al inicio de línea
A+1.56=B	operación con flotantes

El compilador de LEAL requiere conocer previamente las letras que se utilizarán, ya sea como variables enteras o de punto flotante, así como los números de referencia. Para cada variable, el compilador reserva dos ubicaciones y una para cada número de referencia. La variable se declara con el subíndice más alto que aparezca en el programa, lo que implica que automáticamente se declaran todas las anteriores. Por ejemplo, al declarar A(5), se han declarado un total de 6 variables: A(0), A(1), A(2), A(3), A(4) y A(5). De manera similar se declara el número de referencia más alto que aparece en el programa, que aún si no se emplean, debe declararse al menos uno.

Durante la declaración de variables, los sufijos deben ser valores numéricos explícitos, no variables simbólicas.

ENT I(n), FLO A(m), REF(p)	No es permitido
ENT I(5), FLO A(18), REF(2)	Es permitido

La “Facilidad Común” es una regla que permite mapear variables con sufijos mayores al declarado hacia otras variables del mismo tipo declaradas después.

ENT I(3) J(2) K
genera la siguiente correspondencia
I(4) → J(0)
I(5) → J(1)
I(6) → J(2)
I(7) → J(3) = I<(0)

Esto significa que el compilador enlaza automáticamente índices fuera del rango inicial con la siguiente variable declarada. Si aparece I(n) con n > 7, se detecta error. Si se usan sufijos variables (ejemplo: K(L)), el sistema puede redirigir a otra variable. Un caso especial es si L = -3, entonces K(L) se refiere a J(0). Problema: si L < -7, K(L) toma un valor arbitrario sin error detectado, lo que puede causar resultados impredecibles. Por eso se desaconseja el uso de índices negativos. Las mismas reglas aplican tanto a enteros como a flotantes.

El orden de declaración de las variables es arbitrario, pero una vez definido, debe respetarse para que la facilidad común funcione correctamente.

En el siguiente ejemplo se declaran como enteros I(0), J(0), J(1), J(2), K(0), L(0); como flotantes A(0), B(0), B(1), B(2); y como números de referencia (1), (2) y (3).

DECLARACIONES (obligatorias)	
LEAL	encabezado
TITULO	nombre del programa
ENT I J(2) K L	declarar variables enteras y flotante
FLO A B(2)	si no hay variables, se dejan vacias
REF (3)	declarar números de referencia máximo
FIN	fin de programa

La entrada y salida de datos mediante los periféricos tuvo variaciones entre diferentes modelos de las minicomputadoras CID. La CID 201-A leía valores en código Murray desde la cinta perforada/magnética o el teletipo, mientras que la 201-B leía desde teletipos, cintas, tarjetas y discos en diferentes formatos y

códigos. Por ejemplo, para leer una tarjeta de 80 columnas se ejecutaban 80 LCA.

ENTRADA	
LEC variable	Lee un valor desde un periferico
LCA	Lee caracter. Solo LEAL-B

La salida de un programa LEAL se realiza hacia la cinta perforada o el teletipo mediante la instrucción PER. Puede imprimirse en forma aritmética o exponencial base 10, controlado por una instrucción DIG previa al PER. El formato si no se define un DIG es de 4 dígitos enteros y 2 fracionarios en LEAL-A, y 4 dígitos enteros y 2 fracionarios en LEAL-B. En la CID 201-A las salidas son de 5 canales (perforaciones) en la CID 201-B solo se realizaba en 8 canales.

SALIDAS	
LEAL-A	
DIG m	Expon. base 10, m dígitos. Ejmp. m=2: *3.14 e15*
DIG m, n	Aritmética. Ent.Frac, m.n dígitos: 3.14
PER A	m+n+4 espacios
PER J	m+3 espacios
LIN	Nueva línea *cr+lf*
TIT	Imprime Título. No incluye *cr* ni *lf*
LEAL-B (nuevas instrucciones)	
DIG m, n	
PER A	m.n +3 espacios
PER J	m +2 espacios
	m +8 espacios
PCA J	Perforación de caracteres
ESP n	Perfora n espacios
TIT	Imprime Título. Realiza *lf* al final

Las operaciones aritméticas solo se pueden realizar entre variables del mismo tipo. La operación es de izquierda a derecha sin importar el tipo de operador. Debido a esto y el no uso de paréntesis hacía que los programadores necesitaran dividir cálculos complejos en pasos separados, fomentando una programación metódica.

ARITMÉTICAS				
El símbolo “:” se emplea como multiplicación.				
A=C	-A=C	I=K	-I=K	Asignación
A+B=C	-A+B=C	I+J=K	-I+J=K	Suma
A-B=C	-A-B=C	I-J=K	-I-J=K	Resta
A:B=C	-A:B=C	I:J=K	-I:J=K	Multiplicacion
A/B=C	-A/B=C	---	---	División
0.0=C		0=K		
±x=C		±m=K		

EJEMPLO DE EXPRESIÓN ARITMÉTICA	
I+J:K-25 = K	Equivale a K = (I+J)K-25
A+A:B/C-D:0.5 = A	Equivale a A = ((2AB)/C-D)0.5

FUNCIONES	
A SEN = B	Seno, en radianes
A COS = B	Coseno, en radianes
A ACT = B	Arcotangente, en radianes
A LOG = B	Logaritmo en base 10
A LNE = B	Logaritmo en base e
A EXP = B	Exponencial
A RAC = B	Raíz cuadrada
A PEN = J	Parte entera
A FRA = B	Parte fraccionaria
A INV = B	Inverso, B = 1/A
J NOR = B	Normalizar, llevar entero a flotante
J EAB = K	Módulo de una variable entera
J FAB = B	Módulo de una variable flotante
J NEG = K	Negativo

La computadora toma la parte entera de un número como el menor número entero más próximo a él y la parte fraccionaria es el complemento con respecto al número de la parte entera:

NUMERO	ENTERO	FRACCION
3.14	3	14
3	3	0
-3.14	-4	0.86
-0.14	-1	0.9

Los saltos incondicionales se realizan con una instrucción VEA. Los saltos condicionales se expresan como:
EXPRESION ? CONDICION DE SALTO (referencia)

SALTOS	
Incondicionales	
VEA(n)	
Condicionales	
SSP(n)	Salta si >0
SSN(n)	Salta si <0
SSC(n)	Salta si =0
SNP(n)	Salta si ≤0
SNN(n)	Salta si ≥0
SNC(n)	Salta si ≠0

La instrucción CIC permite la ejecución repetida de una serie de instrucciones. Para cada instrucción CIC debe haber una y sólo una instrucción REP (repetir). Antes de comenzar una instrucción cíclica, debe asignarse a la variable de control un valor. Puede haber cuantos ciclos anidados sea necesario, pero es necesario que los números de referencia estén correctamente situados.

CICLO EJEMPLO
VARIABLE/CONSTANTE/EXPRESION = I
(n)CIC
.
.
REP(n) I

En LEAL hay tres tipos de ciclos, definidos por la colocación de las variables o constantes que se ponen tras el número de referencia que sigue a la palabra REP.

CICLO LISTA
(n)CIC
.
.
REP(n) I’J’m’K ;Repetir la cantidad de vars/consts

Este tipo de ciclo consiste en que el lazo se efectúa tantas veces como variables haya. La variable I entra al ciclo con el valor que se le asignó previamente y luego toma los valores de las variables restantes. En este tipo de ciclo todas las variables o constantes van separadas por medio de un apóstrofe.

La variable de control puede ser entera o flotante. En el primer caso, las restantes pueden ser constantes enteras positivas o variables enteras; si la variable de conteo es flotante, el resto pueden ser constantes positivas o variables flotantes. Si hay que hacer el ciclo con una constante negativa, hay que fijar previamente su valor en una variable, y hacer el ciclo con esta variable. Pueden ser tantas variables como sean necesarias.

CICLOS 2DO TIPO
(n)CIC
.
.
REP(n) I, J, K ;Incrementa I en J mientras que I <= K

En este tipo de ciclo el lazo se efectúa primero con el valor que previamente se le a asignado a la variable de conteo I, luego con este valor incrementado en J, repitiéndose mientras sea menor o igual que el valor K.

CICLOS 3ER TIPO	
┌ (n)CIC	
|.	
|. I, m' n	
|.	
└REP(n) I,J' K	Incrementa I en J, K veces

En este tipo de ciclo el lazo se efectúa primero con el valor que previamente se le ha asignado a la variable de conteo I, con este valor incrementado en J en cada ciclo, repitiéndose el ciclo K veces.

Las subrutinas permiten reutilizar bloques de código, lo que hace más eficiente y modular la programación.

SUBRUTINAS. LEAL-A
VAS(n) <const./variables de entrada><variab. de salida>
.
(n)CRU <variables de entrada>
.
FRU(n) <variables de salida> ;Variables emisoras

Pueden no existir variables emisoras y/o receptoras en el programa principal o en la subrutina, pero de existir deben coincidir en el orden y el tipo. En el VAS se separan por (,), pero no así en el CRU y el FRU.

Las variables y valores de entrada se reciben en las variables del CRU. Desde las variables de salida del FRU se emiten los valores a las variables de salida de la función VAS de llamada de la subrutina.

SUBRUTINAS. LEAL-B. SUBINDICE ACOTADO
VAS(n) <variables de entrada> <variables de salida>
.
(n) CRU A(m, n) ;m ≤ n
.
FRU (n)B(m, n)

La instrucción SEP permite especificar los periféricos de entrada y salida que el programa utilizará. Esto es importante en la configuración de las computadoras CID con diferentes dispositivos conectados.

SEPARAR PRIFÉRICOS. LEAL-B
SEP <periférico de entrada>, <periférico de salida>
;los números se toman de la tabla de periféricos
SEP J, K
SEP m, n

Los números de periféricos se toman de una tabla específica, dependiendo de los dispositivos disponibles. Esta funcionalidad simplifica la gestión de los recursos del sistema, como cintas perforadas, discos o teletipos, y permite flexibilidad en la entrada y salida de datos.

MOVIMIENTO DE CARACTERES. LEAL-B	
MDI	mueve a la izq. (se pierden loscarcateres que salen de la cadena) Se llena con espacio por la derecha
MDJ	mueve a la der. (se pierden loscarcateres que salen de la cadena). Se llena con espacio por la izquierda
MDI J(m)	
MID J(K(m))	

ALTO Y FIN	
ALT	detiene la ejecución del programa
FIN	detiene al compilador y la ejecución

Código Interno (CI) del LEAL-B y LEAL-F (para CID 201-B) para conversión entre código de caracteres.

CÓDIGO A

CI	Carácter	ASCII ISO (Octal)	Murray (MTK-2)
0	SP (Espacio)	240	04
1	A	101	30(L)
2	B	102	23(L)
3	C	303	16(L)
4	D	304	22(L)
5	E	305	20(L)
6	F	306	26(L)
7	G	107	13(L)
8	H	110	05(L)
9	I	311	14(L)
10	J	312	32(L)
11	K	113	36(L)
12	L	314	11(L)
13	M	115	07(L)
14	N	116	06(L)
15	O	317	03(L)
16	P	120	15(L)
17	Q	321	35(L)
18	R	322	12(L)
19	S	123	24(L)
20	T	324	01(L)
21	U	125	34(L)
22	V	126	17(L)
23	W	327	31(L)
24	X	330	27(L)
25	Y	131	25(L)
26	Z	132	21(L)
27	£ (Libra)	333	
28	TAB	011	04(sp)
29	Cambio a CODIGO A		02(cr)
30	Cambio a CODIGO B		
31	Nueva Línea	215	10(lf)
32	_ (underscore)	137	
33	@	300	
34	"	042	
35	1/2	243	
36	Fin de texto	003	32(f)
37	%	245	
38	$	246	
39	' (apóstrofo)	047	24(f)
40	(	050	36(f)
41	)	251	11(f)
42	*	252	22(f)
43	+	053	21(f)
44	, <	254, 336	06(f)
45	- (guion)	055	30(f)
46	. >	056, 335	07(f)
47	/	257	27(f)
48	0	060	15(f)
49	1	261	35(f)
50	2	262	31(f)
51	3	063	20(f)
52	4	264	12(f)
53	5	065	01(f)
54	6	066	25(f)
55	7	267	34(f)
56	8	270	14(f)
57	9	071	03(f)
58	:	072	16(f)
59	;	273	
60	$ (variante)	044	
61	=	275	17(f)
62	!	276	
63	?	077	23(f)

CÓDIGO B

CI	Carácter	ASCII ISO (Octal)
0	sp (espacio)	240
1	a	341
2	b	342
3	c	143
4	d	344
5	e	145
6	f	146
7	g	347
8	h	350
9	i	151
10	j	152
11	k	353
12	l	154
13	m	355
14	n	356
15	o	157
16	p	360
17	q	161
18	r	162
19	s	363
20	t	164
21	u	365
22	v	366
23	w	167
24	x	170
25	y	371
26	z	372
28	tab	011
29	Cambio a CODIGO A	
30	Cambio a CODIGO B	
31	Línea	215
32	Carácter no permitido	
34	Start of reading	201
35	Start of text	202
36	End of text	003
37	End of transm	204
40	Retroespacio	207
43	Cancel	012
46	Punch on 2	140
47	Reader excn	175
48	Pare el lector	176
49	Print rest 2	220
50	Prog ind 2	021
51	Prog ind	374
52	File separat	223
53	Print restore	031
54	Unit separat	237
55	Synchron Cicle	226
56	End of transm block	027
57	Prefix code	030
58	Skip restore	231
59	Punch on	232
60	Punch off	033
61	Non print	234
62	Data select	035
63	Record separator	036

Análisis Técnico de las Tablas

Los números como 240, 341, 101 son **códigos octales de 8 bits**. No son decimales.

Para entenderlo: el estándar ASCII/ISO de 7 bits usa valores del 0 al 127. Al añadir un 8º bit (usualmente para paridad o para caracteres extendidos), los valores se representan en octal con 3 dígitos (del 000 al 377).

Ejemplo: La letra A tiene el código ISO 101 en octal, que equivale a 65 en decimal, el estándar ASCII para la 'A'. La a minúscula tiene 341 octal, que equivale a 225 decimal (el código ASCII 97 más el 8º bit de paridad activado, sumando 128).

Nota curiosa: Los símbolos, y . tienen dos códigos ISO asignados (254, 336 y 056, 335). Esto sugiere que la computadora podía emitir dos representaciones distintas dependiendo del contexto (por ejemplo, si se usaba como separador decimal o de lista), o para sortear conflictos de paridad.

La Columna Murray (MTK-2):

Estos también son valores octales, pero de 5 bits (del 00 al 37). Los teletipos usan el código Baudot/MTK-2, que solo tiene 32 combinaciones posibles. Para imprimir más de 32 caracteres, se usan dos "registros" o modos: **Letras (L)** y **Figuras (f)**.

Ejemplo: La letra A y el número 1 comparten el mismo código físico en la cinta (30(L) y 35(f) respectivamente). El teletipo sabe qué imprimir dependiendo de si previamente recibió un carácter de control de "Figuras" o de "Letras".

La Conexión Física (CI 29 y CI 30):

En ambas tablas vemos los comandos **"Cambio A"** y **"Cambio B"**.

Estos son los equivalentes a las teclas que dicen al teletipo que cambie de registro físico antes de imprimir.

Diferencia entre Código A y Código B:

La computadora no usaba ASCII directamente. Usaba su propio **Código Interno (CI)** de 6 bits (0-63).

El Código A era el set principal (mayúsculas y números).

El Código B era un set alternativo (minúsculas y comandos de cinta). El hecho de que las letras minúsculas compartan los mismos códigos CI (1-26) que las mayúsculas indica que la computadora cambiaba internamente entre el "Código A" y el "Código B" para manejar el uso de mayúsculas y minúsculas, algo muy avanzado para un sistema basado en teletipos mecánicos.

Comandos de Periféricos:

Los códigos del 46 al 63 en el Código B (Punch on, Punch off, Reader excn, Pare el lector) demuestran que la CID 201-B estaba diseñada para trabajar con **cintas de papel perforadas**. Estos comandos le decían a la perforadora cuándo empezar a grabar datos y al lector cuándo detenerse.

Ejemplos

El LEAL original para la CID 201-A no poseía opción de comentarios. El LEAL CID 201B incluyó el uso de comentarios mediante el símbolo ";".

```
LEAL ;Encabezado
TITULO SUMA ;Título del programa
ENT A B C ;Declarar flotantes desde periféricos
REF (1) ;Declarar número de referencia máximo
LEC A ;Leer primer número desde el teletipo
LEC B ;Leer segundo número desde el teletipo
(1) A + B = C ;Sumar A y B, y almacenar en C
PER C ;Imprimir el resultado en el teletipo
VEA (1) ; Repetir
FIN
```

```
LEAL
; Encabezado
TITULO COMPORD  ; Título. COMpara y ORDena
ENT A BC        ; Variables de entrada como flotantes
REF (3)                 ;   Declarar   los   números   de
referencia
LEC A                   ; Leer número desde el teletipo
LEC B                   ; Leer número desde el teletipo
(1) A - B = C           ; Restar A - B y almacenar en C
C ? SSN(2)              ; Si C < 0, ir a la instrucción (2)
PER A                   ; Si C >= 0, imprimir A como mayor
PER B                   ; Imprimir B como menor
VEA (3)                 ; Saltar al final
(2) PER B               ; Si C < 0, imprimir B como mayor
PER A                   ; Imprimir A como menor
(3) FIN                 ; Fin del programa
```

```
LEAL
TITULO FACTORIAL
ENT F I N
FLO
REF (1)
1 = I
1 = F
TIT FACTORIAL DE?
LEC N
N + 1 = N
(1) CIC
F : I = F
REP (1) I, 1, N
TIT FACTORIAL=
PER F
FIN
```

```
LEAL
EVALUAR F(X)
ENT
FLO X Y Z
REF (1)
LEC X ; Lectura del valor de X
; Cálculo del denominador: Z = X*X (arctan) + 2
; paso: X * X → AC = X*X
; paso: ACT  → AC = arctan(X*X)
; paso: AC + 2 → Z = 2 + arctan(X*X)
X : X ACT + 2 = Z
; Cálculo del numerador por partes
X COS = Y ; Y = cos(X) - Y
X SEN / Y + X EXP = Y ; Y = (sen(X) / Y) + exp(X)
X LNE RAC + Y / Z = Y ; Y = (ln(X) RAC + Y) / Z
PER Y
FIN
```

```
LEAL
SOL.  ECUACION DE SEGUNDO GRADO
ENT
FLO A B C D
REF (2)
; LECTURA DE COEFICIENTES
LEC A
LEC B
LEC C
B : B = D ; D = B^2 ; DISCRIMINANTE: D = B^2 - 4AC
-4.0 : A : C + D = C ; C = -4AC + D   (C CONTIENE EL
DISCRIMINANTE)
; ANÁLISIS DEL DISCRIMINANTE
C ? SSN (1) ; SI C < 0 → NO HAY SOLUCIÓN REAL, FIN
C ? SSC (2) ; SI C = 0 → RAÍZ DOBLE
; RAÍCES COMPLEJAS (DISCRIMINANTE NEGATIVO PERO NO LLEGÓ POR
C?)
C RAC = C ; C = SQRT(C)  (RAÍZ DEL DISCRIMINANTE)
-B - C : 0.5 / A = D ; D = (-B - C) * 0.5 / A
PER D ; SEGUNDA RAÍZ (A TRAVÉS DE (2))
(2) -B + C : 0.5 / A = B ; B = (-B + C) * 0.5 / A
PER B
(1) FIN
```

```
LEAL
HALLAR AREA
ENT
FLO A B C D
REF (1)
TIT AREA DE UN POLIGONO
LIN
; LECTURA DE CUATRO VALORES
TIT A?
LEC A
TIT B?
LEC B
TIT C?
LEC C
TIT D?
LEC D
; FÓRMULA: D = B/2 + C*A*D
; (EVALUACIÓN ESTRICTA DE IZQUIERDA A DERECHA)
; PASO 1: B / 2 → AC = B/2
; PASO 2: AC + C → AC = B/2 + C
; PASO 3: AC * A → AC = (B/2 + C) * A
; PASO 4: AC * D → AC = (B/2 + C) * A * D
; DESTINO: D
B / 2 + C : A : D = D
TIT AREA:
LIN
PER D   ; IMPRIME EL RESULTADO
FIN
```

```
LEAL
EVALUAR F(X)
ENT
FLO X Y Z
REF (1)
;  LECTURA DEL VALOR DE X
LEC X
;  CÁLCULO DEL DENOMINADOR: Z = X*X (ARCTAN) + 2
; PASO: X * X → AC = X*X
; PASO: ACT  → AC = ARCTAN(X*X)
; PASO: AC + 2 → Z = 2 + ARCTAN(X*X)
X : X ACT + 2 = Z
;  CÁLCULO DEL NUMERADOR POR PARTES
; Y = COS(X) - Y     (Y INICIALMENTE 0, PERO SE USA LUEGO)
X COS = Y
; Y = (SEN(X) / Y) + EXP(X)
X SEN / Y + X EXP = Y
; Y = (LN(X) RAC + Y) / Z
X LNE RAC + Y / Z = Y
PER Y
FIN
```

```
LEAL
SUM POSITIV,SUM CUAD D NEGAT,CONTADORES
ENT I J
FLO A B X
REF (3)
; INICIALIZACIÓN DE ACUMULADORES
0.0 = A; A: SUMA DE POSITIVOS
0.0 = B; B: SUMA DE CUADRADOS DE NEGATIVOS
0 = I; I: CONTADOR DE POSITIVOS
0 = J; J: CONTADOR DE NEGATIVOS
; BUCLE DE LECTURA
(3)LEC X; LEE EL SIGUIENTE NÚMERO
X ? SSC (1); SI X = 0, FIN DE LA LISTA
X ? SSN (2); SI X < 0, ACUMULA EN B
; X > 0:
X + A = A
I + 1 = I
VEA (3)
(2)X : X + B = B; SUMA DE CUADRADOS DE NEGATIVOS
J + 1 = J
VEA (3)
; SALIDA DE RESULTADOS
(1)PER A
PER B
PER I
PER J
I + J = J; J = TOTAL DE NÚMEROS LEÍDOS
PER J
FIN
```

```
LEAL
SUMA DE CUADRADOS DE 5 NUMEROS (LECTURA CON CICLO TIPO 2)
ENT I
FLO X(5)
REF (2)
TIT SUMA DE CUADRADOS DE 5 NUMEROS
LIN
;  LECTURA USANDO CICLO TIPO 2
0 = I
(1)CIC
TIT NUM?
LEC X(I)
REP (1) I, 1, 5
;  SUMA DE CUADRADOS
0.0 = X
1 = I
(2)CIC
X(I) : X(I) + X = X
REP (2) I, 1, 5
PER X
FIN
```

```
LEAL
EVAL POLINOMIO EN VALORES ARBITRARIOS POR HORNER
ENT N I
FLO A(100) D X
REF (4)
;  LECTURA DEL GRADO
LEC N
;  LECTURA DE LOS COEFICIENTES A(0)..A(N)
0 = I
(1)LEC A(I)
I + 1 = I
I - N - 1 ? SNC (1); REPITE HASTA LEER N+1 COEFICIENTES
;  PRIMER VALOR DE X
-8.5 = X
;  CICLO LISTA: EVALÚA EL POLINOMIO PARA X = -8.5, 2, 3.87
(2)CIC
0.0 = D; D = 0 (ACUMULADOR)
0 = I; I = 0
(3)D + A(I) = D; D = D + A(I)
N - I ? SSC (4); SI I = N, TERMINA LA EVALUACIÓN
D : X = D; D = D * X
I + 1 = I
VEA (3)
(4)PER D; IMPRIME EL VALOR DEL POLINOMIO
REP (2) X'2'3.87; REPITE PARA CADA VALOR DE X
FIN
```

```
LEAL
HALLAR LA TRANSPUESTA DE UNA MATRIZ
ENT I J M N P
FLO A(500)
REF (3)
;  LECTURA DE DIMENSIONES
LEC M; FILAS
LEC N; COLUMNAS
;  P = M * N  (TOTAL DE ELEMENTOS)
M : N = P
;  LECTURA DE LA MATRIZ A
1 = I
(1)CIC
LEC A(I)
REP (1) I, 1'P; RECORRE I DE 1 A P
;  DOBLE CICLO: J = COLUMNAS, I = FILAS
```

```
1 = J
(2)CIC
0 = I
(3)CIC
I + J = P; P = (I,J) LINEALIZADO
PER A(P); IMPRIME A(I,J)
REP (3) I, N' M; RECORRE I DE 1 A M
REP (2) J, 1'N; RECORRE J DE 1 A N
FIN
```

```
LEAL
MULTIPLICACION DE DOS MATRICES
ENT H I J K L M N S
FLO A(250) B(250)
REF (5)
;  LECTURA DE DIMENSIONES
LEC M; FILAS DE A
LEC N; COLUMNAS DE A / FILAS DE B
LEC K; COLUMNAS DE B
;  H = M * N  (TOTAL DE ELEMENTOS DE A)
N : M = H
;  LECTURA DE LA MATRIZ A POR FILAS
1 = I
(1)CIC
LEC A(I)
REP (1) I, 1'H; RECORRE I DESDE 1 HASTA H
;  H = M * K  (TOTAL DE ELEMENTOS DE B)
M : K = H
;  LECTURA DE LA MATRIZ B POR FILAS
1 = I
(2)CIC
LEC B(I)
REP (2) I, 1'H
;  TRIPLES CICLOS ANIDADOS: I (FILAS DE A), H (FILAS DE
B), J (COLUMNAS DE B)
1 = I
(3)CIC
1 = H
(4)CIC
0.0 = B; ACUMULADOR DEL PRODUCTO PUNTO
0 = J
(5)CIC
I + J = L; L = ÍNDICE DE A(L)
K : J + H = S; S = ÍNDICE DE B(S)
A(L) : B(S) + B = B; ACUMULA EL PRODUCTO
REP (5) J, 1'M; J DE 1 A M
PER B; IMPRIME EL ELEMENTO (I,H) DEL PRODUCTO
REP (4) H, 1'K; H DE 1 A K
REP (3) I, M' K; I DE 1 A K
FIN
```

```
LEAL
CALCULO DE UNA INTEGRAL POR LA FORMA DE SIMPSON
ENT N I
FLO A B C H S
REF (3)
;  LECTURA DE DATOS
LEC N; NÚMERO DE INTERVALOS
LEC A; EXTREMO INFERIOR
LEC B; EXTREMO SUPERIOR
;  CÁLCULO DEL PASO H = (B - A) / N
N NOR = H; CONVIERTE N A FLOTANTE
B - A / H = H; H = (B - A) / N
;  PRIMERA EVALUACIÓN DE LA FUNCIÓN EN A
VAS (1) A, C; C = F(A)
0 = I; CONTADOR DE INTERVALOS RECORRIDOS
C = S; S = F(A)  (ACUMULADOR DE LA SUMA)
;  BUCLE PRINCIPAL DE SIMPSON
(3)A + H = A; AVANZA AL SIGUIENTE PUNTO
I + 1 = I; I++
VAS (1) A, C; C = F(A)
C : 4 + S = S; S += 4*F(A)
I - N + 1 ? SSC (2); SI I = N-1, SALTA A LA ÚLTIMA
ITERACIÓN
A + H = A
I + 1 = I
VAS (1) A, C
C : 2 + S = S; S += 2*F(A)
VEA (3); REPITE
;  TRAMO FINAL DE LA INTEGRAL
(2)A + H = A
VAS (1) A, C
C + S : H / 6 = S; S = (C + S) * H / 6
PER S
ALT
;  SUBRUTINA DE LA FUNCIÓN A INTEGRAR: F(A)
; F(A) = (A*A - 4*A) RAC  /  (1.0 / A) ACT
(1)CRU A; RECIBE A COMO ENTRADA
A : A - 4 : A RAC = C; C = SQRT(A*A - 4*A)
1.0 / A ACT : C = C; C = C * ATAN(1/A)
FRU (1) C; DEVUELVE C COMO SALIDA
FIN
```

```
LEAL
CALCULO DE LA MEDIA Y LA DESVIACION TIPICA
ENT I N
FLO X(400) A B Y
REF (2)
;  LECTURA DEL NÚMERO DE ELEMENTOS
LEC N
;  LECTURA Y SUMA DE LOS N ELEMENTOS
1 = I
0.0 = X; X ACUMULA LA SUMA
(1)CIC
LEC X(I)
X(I) + X = X; X += X(I)
REP (1) I, 1, N
;  MEDIA: A = X / N
N NOR = A; CONVIERTE N A FLOTANTE
X / A = X; X = X / N  (MEDIA)
PER X; IMPRIME LA MEDIA
;  CÁLCULO DE LA DESVIACIÓN TÍPICA
1 = I
0.0 = Y; Y ACUMULA LA SUMA DE (XI - X)^2
(2)CIC
X(I) - X = B; B = XI - MEDIA
B : B + Y = Y; Y += B^2
REP (2) I, 1, N
;  RAÍZ CUADRADA DE (Y / N)
Y / A RAC = Y; Y = SQRT(Y / N)
PER Y
FIN
```

```
LEAL
HALLAR V
ENT
FLO D L K F V
REF (1)
;  VALORES INICIALES
30.0 = D
270.0 = L
;  CICLO LISTA: L RECORRE 270, 275, 280, 285, 290
(1)CIC
6.3 : D / L = K; K = (6.3 * D) / L
-K EXP = F; F = EXP(-K)
K EXP = K; K = EXP(K)
K + F = V; V = EXP(K) + EXP(-K) (DENOMINADOR)
K - F / V : 1.8 : L = V; V = (EXP(K) - EXP(-K)) / V * 1.8
* L
PER V
D + 1 = D; INCREMENTA D
REP (1) L, 5, 290; L DE 270 A 290 PASO 5
FIN
```

```
LEAL
HALLAR LA TRANSPUESTA DE UNA MATRIZ
ENT I J M N P
```

```
FLO A(500)
REF (3)
;  LECTURA DE DIMENSIONES
LEC M; FILAS
LEC N; COLUMNAS
;  P = M * N  (TOTAL DE ELEMENTOS)
M : N = P
;  LECTURA DE LA MATRIZ A
1 = I
(1)CIC
LEC A(I)
REP (1) I, 1'P; RECORRE I DE 1 A P
;  DOBLE CICLO: J = COLUMNAS, I = FILAS
1 = J
(2)CIC
0 = I
(3)CIC
I + J = P; P = (I,J) LINEALIZADO
PER A(P); IMPRIME A(I,J)
REP (3) I, N' M; RECORRE I DE 1 A M
REP (2) J, 1'N; RECORRE J DE 1 A N
FIN
```